SIMATS ENGINEERING

ASSESSMENT TOOL 2 (AT2) — COURSE OUTCOME 2 (CO2)

Course Name:	Artificial Intelligence	**Course Code:**	CSA17
Student Name:	**Shaik Reehan**	**Register Number:**	**192565071**
Duration:	1 Hour	**Total Marks:**	50 (Weightage: 50%)
Course Outcome:	CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)		

Code of Conduct & Student Declaration:

I, **Shaik Reehan (Reg No: 192565071)**, certify that this submission is my original work and that I have adhered to the academic integrity guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of Student: Shaik Reehan **Date of Submission:** October 21, 2026

ANNEXURE A: SCENARIO BASED ASSIGNMENT

Scenario 1: AI-Powered Drone Delivery in Dynamic Flood-Affected Regions

Context: A government deploys autonomous drones to deliver emergency medical kits across flood-hit terrain. The operational environment is highly dynamic: roads and landing sites become flooded unpredictably, adverse weather (gale winds, rain) alters battery power consumption, and the drone has straight-line Euclidean distance estimates, partial real-time hazard updates, and variable movement costs.

i. Greedy Best-First Search Behavior, Strengths & Weaknesses under Uncertainty

- **Behavior:** Greedy Best-First Search selects the next navigation waypoint strictly minimizing estimated heuristic cost to destination: $f(n) = h(n)$ (straight-line distance). It completely ignores the accumulated historical cost $g(n)$ incurred to reach node n.
- **Strengths:**
  1. *Rapid Decision Velocity:* In unblocked open air corridors, it expands very few nodes and immediately targets the destination heading.
  2. *Low Computational Footprint:* Minimal memory and processing overhead, suitable for low-power onboard drone microcontrollers.
- **Weaknesses under Uncertainty:**
  1. *Susceptibility to Dead-Ends (False Leads):* It flies directly into flooded mountain valleys or blocked corridors simply because their Euclidean coordinate is closer to target, forcing severe mid-air backtracking.
  2. *Cost Blindness & Safety Risk:* It treats a hazardous high-wind storm corridor and a calm detour identically if their straight-line distances match, frequently exhausting battery reserves prematurely.
  3. *Lack of Optimality:* Non-optimal and incomplete in infinite or cyclic search graphs under dynamic changes.

ii. A* Search Handling: Balancing Path Cost and Heuristic

- **Cost-Heuristic Balancing:** A* evaluates nodes using the evaluation function:

$$f(n) = g(n) + h(n)$$

  where $g(n)$ represents the true cumulative travel cost from start (integrating wind resistance, rain severity, and altitude climbs) and $h(n)$ estimates the remaining flight distance to target.
- **Mechanism under Flood Conditions:** If a direct flight corridor experiences storm headwinds, its accumulated edge cost $c(u, v)$ rises, driving up $g(n)$. When $f(n)$ exceeds an alternate path, A* naturally redirects exploration toward a longer but calmer and safer detour, balancing battery conservation against remaining distance.
- **Optimality Guarantee:** A* guarantees finding the lowest-cost, safest path provided the heuristic $h(n)$ is admissible ($h(n) \le h^*(n)$) and consistent ($h(n) \le c(n, a, n') + h(n')$).

iii. Analysis of Heuristic Accuracy on Both Algorithms

Heuristic Accuracy	Impact on Greedy Best-First Search	Impact on A* Search
Perfect Heuristic ($h = h^*$)	Proceeds directly along the optimal trajectory without expanding any redundant nodes.	Expands solely nodes along the optimal path with zero search backtracking.

Heuristic Accuracy	Impact on Greedy Best-First Search	Impact on A* Search
Admissible Underestimate ($h \leq h^*$)	Frequently misled into local dead-ends that appear geometrically closer.	Guarantees mathematical optimality; lower h increases node expansions toward Dijkstra.
Overestimate ($h > h^*$)	Highly erratic; prone to severe oscillation and trapped trajectories.	Forfeits optimality guarantee; may return dangerous or battery-depleting sub-optimal paths.

Straight-Line Distance in Flooding: Straight-line Euclidean distance is strictly admissible ($h \leq h^*$) since straight flight is the theoretical minimum. However, extensive flood blockages make the true path h^* significantly longer, widening the error gap $h^* - h$ and forcing A* to expand more nodes.

iv. Impact of Dynamic Changes (Blocked Paths) on Search Performance

- **Invalidation of Offline Trajectories:** In a dynamic flood environment, static paths precalculated before takeoff fail immediately when radar sensors detect newly submerged landmarks or fallen powerlines.
- **Replanning Overhead:** Classical A* must discard its search tree and restart from scratch ($O(b^d)$), causing dangerous drone hover delays that deplete emergency battery reserves.
- **Incremental Repair:** Real-time algorithms like **D* Lite** or **Lifelong Planning A* (LPA*)** maintain prior search trees, updating only the cost values of edges affected by newly discovered blockages, replanning new routes in milliseconds.

v. Recommended Algorithm & Justification

Recommendation: D* Lite (Dynamic A*) with a Safety-Weighted Cost Model.

Justification:

1. *Real-Time Responsiveness:* D* Lite updates flight trajectories incrementally in milliseconds upon receiving partial telemetry, avoiding full re-search overhead.
2. *Safety & Battery Optimality:* By tracking true energy cost $g(n)$, it guarantees the drone selects routes that do not deplete battery reserves in high-wind zones.
3. *Completeness:* Systematically finds an alternate safe delivery route if any unobstructed path exists.

Scenario 2: Smart City Traffic Signal Timing Optimization

Context: A municipal AI management platform optimizes traffic signal timings across hundreds of interconnected urban intersections. Goals: minimize waiting time, fuel burn, and traffic queue lengths under dynamically shifting flow patterns.

Problem Formulation as an Optimization Problem

- **State Representation:** Configuration vector $S = (\theta_1, \theta_2, ..., \theta_N)$ for all N intersections, where each $\theta_i = (t_{green}, t_{yellow}, t_{red}, \varphi_{offset})$ represents phase splits and arterial cycle offsets.
- **Objective Function (To Minimize):**

$$J(S) = w_1 \sum_i Delay_i(S) + w_2 \sum_i FuelConsumption_i(S) + w_3 \sum_i QueueLength_i(S)$$

- **Why Traditional Search is Inefficient:**
  1. *Combinatorial Explosion:* With 200 intersections and continuous/discrete timing parameters, the state space exceeds 10^{150} states, rendering BFS/DFS/A* computationally intractable.
  2. *No Explicit Goal State:* The system seeks an optimal configuration minimizing $J(S)$, not a step-by-step path between start and end nodes.
  3. *Dynamic Non-Stationary Landscape:* Traffic surges shift the cost function continuously over time.

i. Hill Climbing Operation & Limitations

- **Operation:** Starts from current timing configuration S. Generates neighboring timing plans by adjusting green splits by small increments (± 3 to 5 seconds). Greedily transitions to the neighbor yielding the steepest reduction in $J(S)$. Halts when no neighbor improves total cost.
- **Limitations:** Completely myopic. It strictly rejects any timing modification that temporarily increases delay, making it incapable of escaping suboptimal local configurations.

ii. Real-World Interpretation of Search Landscape Obstacles

Landscape Obstacle	Theoretical Definition	Real-World Traffic System Interpretation
Local Maxima / Minima	A state superior to all immediate neighbors but inferior to global optimum.	A timing plan where locally optimizing intersection A causes gridlock at downstream intersection B. Single adjustments increase total city delay, trapping the search.
Plateaus	A flat region of the search space where all neighbor states yield identical objective values.	Off-peak night hours where shifting green time by ± *2*s causes zero change in empty vehicle queues. Hill climbing wanders in an aimless random walk.
Ridges	A sloping narrow crest where navigating requires simultaneous coordinated steps.	Arterial "Green Wave" synchronization. Progression requires simultaneous coordinated offset changes across 5 consecutive lights; single-light changes fail.

iii. Overcoming Limitations via Simulated Annealing & Genetic Algorithms

- **Simulated Annealing (SA):** Introduces stochastic exploration. Uphill moves (worse timing plans with $\Delta J > 0$) are accepted with probability:

  $$P(accept) = e^{-\Delta J / T}$$

  At high temperature T (initial peak hours), the system accepts larger trade-offs to escape poor local traps. As $T \to 0$, it cools into fine-tuned greedy exploitation.
- **Genetic Algorithms (GA):** Maintains a population of diverse timing plans (chromosomes).
  - *Crossover:* Exchanges coordinated corridor timing blocks between successful city districts.
  - *Mutation:* Introduces random timing perturbations, injecting diversity to jump across ridges and plateaus.
  - *Selection:* Propagates low-delay, low-emission signal configurations to subsequent generations.

iv. Recommended Approach & Justification

Recommendation: Hybrid Memetic Algorithm (Genetic Algorithm with Local Hill Climbing Refinement).

Justification:

1. *City-Wide Scalability:* GA evaluates hundreds of intersections concurrently using parallel fitness evaluations across sub-networks.
2. *Fast Convergence:* Local hill climbing quickly optimizes individual intersection phase splits, while GA provides global arterial coordination.
3. *Dynamic Adaptability:* Rolling populations adapt seamlessly to transition periods (e.g. school closing, rush hours) by re-injecting historical seed chromosomes.

Scenario 3: Autonomous Mars Rover in Unknown Terrain

Context: An exploration rover navigates uncharted Martian terrain to sample geological targets, avoid craters and wheel-trapping sand dunes, and make real-time exploration choices without pre-existing topographical maps.

i. Why Online Search is Required Instead of Offline Search

- **Offline Search Presumption:** Classical offline algorithms (e.g., standard A*, Dijkstra) calculate a complete end-to-end plan from Start to Goal before taking a single physical action. This requires a 100% complete, static, and deterministic environmental model.
- **The Martian Reality:** High-resolution surface geometry, boulder distributions, and loose regolith soil density are completely unknown prior to arrival. An offline plan generated from low-resolution satellite maps would inevitably drive the rover into an unmapped crevasse.
- **Online Search Requirement:** The rover must interleave *local perception, internal mapping, planning, and execution*, taking physical actions in the world to reveal new information.

ii. Analysis of Partial Observability & Dynamic Environment Challenges

- **Partial Observability:** Onboard stereoscopic Hazcams and LIDAR have a range horizon of 5 to 15 meters. Occlusions behind cliffs and rocks hide terrain hazards. The rover must make navigation commitments while in a state of perceptual ignorance about distant paths.
- **Dynamic & Non-Deterministic Realities:**

1. *Traction Slippage:* Commanding a 1-meter forward drive on loose Martian dunes may produce only 0.4 meters of progress with severe lateral drift.
2. *Atmospheric Hazards:* Wind-blown dust storms reduce solar panel power generation, forcing sudden replanning to safe sleep zones.

iii. Internal Model Construction and Updating

- **2.5D Digital Elevation & Occupancy Grid:** The rover maintains an evolving spatial coordinate map M storing elevation, terrain gradient, and cell classification (Passable, Hazard, Unknown).
- **Bayesian Belief Updating:** As new stereo image depth maps arrive, cell traversability probabilities are updated recursively:

$$P(Hazard \mid Percept_{1:t}) \propto P(Percept_t \mid Hazard) \times P(Hazard \mid Percept_{1:t-1})$$

- **Traversability Cost Function:** Edge travel cost combines slope grade, surface roughness, and wheel sinkage probability derived from motor current telemetry.

iv. Comparison of Online Search with Uninformed & Informed Strategies

Search Strategy	Exploration Mechanics	Performance & Viability on Mars
Uninformed Online (DFS / Random Walk)	Moves arbitrarily or follows walls blindly without goal direction.	**Unviable:** Extremely high risk of battery exhaustion; easily trapped in large craters; lacks goal orientation.
Informed Offline (Classical A*)	Precalculates entire route based on initial prior assumptions.	**Fails:** Cannot handle surprise obstacles; invalidates entire path upon encountering first unmapped boulder.
Informed Online (LRTA* / Real-Time A*)	Uses heuristic $h(n)$ to target goal; updates heuristic values of visited cells to prevent cycles.	**Optimal:** Guarantees goal reachability without infinite loops; bounded computation fits onboard CPU limits.

v. Recommended Approach & Justification

Recommendation: Learning Real-Time A* (LRTA*) with a Hybrid Deliberative-Reactive Architecture.

Justification:

1. *Completeness & Loop Avoidance:* LRTA* updates its heuristic estimates whenever it encounters dead-ends ($h(u) \leftarrow max(h(u), c(u, v) + h(v))$), ensuring it escapes cul-de-sacs without looping.
2. *Bounded Computation:* Generates actions within strict time bounds matching rover movement cycles.
3. *Reactive Safety Reflex:* A low-level reactive layer monitors IMU accelerometers and wheel encoders to halt motors instantaneously if slippage exceeds 20% or tilt exceeds 15°.

Scenario 4: University Exam Timetable Generation (CSP)

Context: A university schedules semester examinations across hundreds of courses and thousands of students. Hard constraints prevent student conflicts and hall overflows; soft constraints govern faculty workload and student spacing.

i. Problem Formulation as a Constraint Satisfaction Problem (CSP)

- **Variables:** Set of all course exams:

$$X = \{E_1, E_2, ..., E_N\}$$

- **Domains:** Set of valid assignment pairs of exam timeslot t and examination hall h:

$$D(E_i) = \{ (t, h) \mid t \in \{Day_1..Day_K, Slot_1..Slot_3\}, h \in \{Hall_1..Hall_M\} \}$$

- **Constraints:**
  1. **Student Conflict Constraint (Hard Binary):** No student can write two exams simultaneously:
  $$Enrolled(E_i) \cap Enrolled(E_j) \neq \emptyset \Rightarrow TimeSlot(E_i) \neq TimeSlot(E_j)$$
  2. **Hall Capacity Constraint (Hard Resource):** Room capacity must not be exceeded:
  $$EnrollmentCount(E_i) \leq Capacity(Hall(E_i))$$
  3. **Prerequisite Precedence (Hard Binary):** Foundational courses must be examined before advanced electives:
  $$TimeSlot(E_{core}) < TimeSlot(E_{advanced})$$
  4. **Special Accommodations (Hard Unary):** Students requiring accessibility or extended time must be placed in designated accessible ground-floor halls.
  5. **Faculty Availability (Hard Resource):** Invigilators assigned to at most one exam per slot.

ii. Backtracking Search Operation in Exam Timetabling

- Backtracking Search is a recursive Depth-First Search algorithm tailored for CSPs.
- **Operation:**
  1. Picks an unassigned exam E_i.
  2. Iterates through available *(timeslot, hall)* pairs in its domain.
  3. Checks assignment against all constraints with previously scheduled exams.
  4. If no constraints are violated, assigns E_i and recursively calls search for E_{i+1}.
  5. If a conflict occurs, prunes the branch immediately, resets E_i, and backtracks to try alternative timeslots.

iii. Efficiency Gains via Constraint Propagation (Forward Checking & AC-3)

- **Forward Checking (FC):** Whenever exam E_i is assigned timeslot t, FC immediately scans all unassigned exams E_j that share enrolled students and removes timeslot t from their domain. If any unassigned exam's domain becomes empty, it halts and backtracks immediately, preventing deep unproductive search branches.
- **Arc Consistency (AC-3):** AC-3 ensures that for every value in the domain of E_i, there exists at least one legal supporting value in the domain of E_j. It propagates constraints across the entire constraint graph, pruning conflicting timeslots globally before branching, reducing the search space exponentially.

iv. Scalability Challenges & Recommended Strategy

Optimization Component	Heuristic & Implementation Justification
Variable Ordering	**Minimum Remaining Values (MRV) + Degree Heuristic:** Selects the exam with the fewest legal timeslots left ("fail-first" principle). Ties are broken by Degree Heuristic (selecting exams with the highest number of student conflicts).
Value Ordering	**Least Constraining Value (LCV):** Picks the timeslot/hall that leaves the maximum flexibility (maximum remaining legal slots) for all neighboring exams.
Ultra-Large Scale Strategy	**Min-Conflicts Local Search:** For campus-wide problems (>1,000 exams), backtracking can become slow. Min-Conflicts starts with an initial complete timetable and iteratively reassigns conflicting exams to slots minimizing violations, solving massive instances in near-linear time.

Scenario 5: Strategic Game AI for Real-Time Decision Making

Context: A gaming company designs an AI commander for a real-time tactical strategy game. The AI battles human players under strict time deadlines (e.g. 500 ms per turn) across complex game states with massive branching factors.

i. Minimax Algorithm Modeling

- **Zero-Sum Adversarial Formulation:** The game is modeled as a 2-player zero-sum game between **MAX** (the AI agent seeking to maximize game score) and **MIN** (the human opponent seeking to minimize AI advantage).
- **Recursive Value Function:**

$$V(s) = \begin{cases} f(s) & \text{if Terminal/Cutoff(s)} \\ \max_a V(\text{Result}(s, a)) & \text{if Player(s) = MAX} \\ \min_a V(\text{Result}(s, a)) & \text{if Player(s) = MIN} \end{cases}$$

  The AI assumes the human player always plays rationally to minimize MAX's outcome.

ii. Computational Challenges of Large Game Trees

- **Combinatorial Explosion:** In strategy games, branching factor $b \geq 40$ moves per turn, and depth $d \geq 20$ plies. The total search tree size $O(b^d)$ exceeds 10^{32} game states.
- **Strict Real-Time Latency Limits:** Evaluating the full game tree is impossible within strict latency limits (500 ms). Exhaustive search produces CPU freezing, violating gameplay quality.

iii. Alpha-Beta Pruning Efficiency & Reasoning

- **Pruning Mechanism:** Maintains two bounds along the search path:
  - α: The highest utility value found so far for MAX along the path (initially $-\infty$).
  - β: The lowest utility value found so far for MIN along the path (initially $+\infty$).
- **Cutoff Rules:**

1. *Alpha Cutoff:* At a MIN node, if candidate value $v \le \alpha$, search halts. Reason: MAX already has a proven better choice ($\ge \alpha$) on an alternate path; MAX will never allow the game into this MIN branch.
2. *Beta Cutoff:* At a MAX node, if candidate value $v \ge \beta$, search halts. Reason: MIN already has a lower choice ($\le \beta$) elsewhere; MIN will never permit this state.

- **Complexity Improvement:** With optimal move ordering, Alpha-Beta pruning reduces complexity from $O(b^d)$ to $O(b^{d/2})$, effectively **doubling the reachable search depth** within the same CPU time budget.

iv. Design of a Multi-Factor Heuristic Evaluation Function

Because search must be truncated at depth d_{limit}, non-terminal leaves are evaluated by a multi-attribute evaluation function:

$$\boldsymbol{Eval(s) = w_1\,\Delta Material + w_2\,TerritoryControl + w_3\,UnitMobility + w_4\,BaseSafety + w_5\,EconomicOutput}$$

Evaluation Feature	Weight	Strategic Justification
ΔMaterial Balance	0.35	Direct indicator of military force strength and combat unit survival.
Territory / Map Control	0.25	Choke-point dominance, forward outpost control, and vision radius.
Unit Mobility	0.15	Number of legal tactical moves, preventing unit entrapment.
Base / Commander Safety	0.15	Defense perimeter integrity; penalizes exposed command centers.
Economic Harvester Output	0.10	Resource income rate to fund unit replacements and upgrades.

v. Strategy for Balancing Optimality & Real-Time Performance

- **Iterative Deepening Search (IDS):** Searches depth 1, then depth 2, depth 3... progressively. If the 500 ms timer triggers, the algorithm halts and immediately returns the best move from the last fully completed ply (anytime algorithm guarantee).
- **Transposition Tables (Zobrist Hashing):** Stores evaluated positions in a hash table. If identical board states occur via transposed move orders, cached evaluations are used instantly, eliminating redundant expansions.
- **Move Ordering via Killer Moves:** Evaluates moves that caused beta cutoffs in sibling branches first, driving pruning efficiency toward the theoretical ideal $O(b^{d/2})$.
- **Quiescence Search:** Extends search at leaf nodes where volatile tactical engagements (unit battles, base assaults) are active, eliminating the "horizon effect" error.